Milestone 1 Report — CS265 LLM Inference Project

Stephen Pasco
3/23/2026
Credits: Gemini, Grammerly

Project Overview

Name	CS265-llm-starter
Languages	C++17, Python
Frameworks	CUDA
Description	From-scratch implementation of Llama 3 8B inference — BPE tokenizer, binary weight loader, GPU operators, and tiled GEMM kernel. No ML framework dependencies at runtime.

Quick Start

```
# Build and run the main binary
make run

# Build and run all 7 tests (requires model weights in ./assets/llama3/)
make tests
for i in 1 2 3 4 5 6 7; do ./bin/tests $i; done

# Debug build
make BUILD=debug
```

Introduction

This project implements the Llama 3 8B inference pipeline from scratch in C++17 and CUDA. It covers every stage from text tokenization through GPU-accelerated matrix multiplication.

The codebase follows a layered architecture:

1. A **BPE tokenizer** encodes input text into token IDs.
2. A **binary weight loader** reads model parameters from a custom dump format.
3. A **double-buffered tiled GEMM kernel** executes linear layers on an NVIDIA GPU.

All 7 milestone 1 tests pass on a GCP T4 instance, validating the tokenizer, embedding extraction, and matmul against reference binary fixtures.

Problems Tackled

- **Tokenization** — Convert raw text to token IDs using Byte Pair Encoding with special token handling, base64-encoded vocabulary, and a greedy merge loop matching the reference HuggingFace tokenizer exactly.
- **Detokenization** — Convert token IDs back to text, reversing BPE while handling special tokens, byte-level fallback characters, and UTF-8 boundaries.
- **Weight Loading** — Read Llama 3 8B's multi-gigabyte weights from a custom binary dump format (256-byte header + BF16 payload), convert BF16 to FP32 at load time, and extract per-token embedding vectors on demand.
- **Weight Conversion Pipeline** — Download Llama 3 weights from HuggingFace (safetensors) and convert them to our custom binary format for efficient memory-mapping.
- **Float Conversion** — Bit-accurate BF16-to-FP32 and FP16-to-FP32 converters that reconstruct sign, exponent, and mantissa fields without hardware support or compiler intrinsics.
- **GPU Matrix Multiplication** — High-performance CUDA GEMM kernel for Llama 3's linear layers, using shared-memory tiling, double buffering, `float4` vectorized loads, and bank-conflict-free padding.
- **CPU Fallback** — CPU matmul path with cache-friendly (i,k,j) loop ordering so the project builds and runs without an NVIDIA GPU.
- **Test Bridge** — Wire tokenizer, weight loader, and matmul into the professor's read-only test harness via a bridge class, satisfying 7 graded tests against binary reference fixtures.

Technical Description

1. BPE Tokenization

a) Problem framing. Llama 3 expects integer token IDs, not raw text. The tokenizer must split UTF-8 strings into subword units matching the reference HuggingFace tokenizer exactly. The vocabulary is base64-encoded in `tokenizer.model` with merge priority ranks, and 256 special control tokens live outside the normal vocab range.

b) High-level solution. Two hash maps: `rank` (`unordered_map<string, int>`) maps tokens to merge priority, `id2tok` (vector) maps IDs back to strings. Encoding scans for special tokens first (longest-match), then splits remaining text into bytes and runs a greedy merge loop. O(n^2) per chunk, acceptable since chunks are typically short. Decoding is O(n) table lookup.

c) Deeper details.

```
encode(text):
  for each position i:
    scan specials_sorted[] for longest match → emit special ID
    else: extract chunk to next special boundary
      encode_chunk(chunk):
        split into bytes: ["H","e","l","l","o"]
        while |toks| > 1:
          find adjacent pair with min rank[concat]
          if none: break
          merge toks[i] += toks[i+1]; erase toks[i+1]
        map final tokens → rank IDs
```

2. Binary Weight Loading

a) Problem framing. Llama 3 8B has ~8B parameters across hundreds of tensor files. Safetensors requires JSON parsing and variable-length headers. We need a format with minimal parsing that supports on-demand row extraction from the 128k × 4096 embedding table.

b) High-level solution. Custom dump format: fixed 280-byte header + raw BF16 payload. The loader reads the file into a `vector<uint8_t>`, parses the header with pointer arithmetic, and converts BF16→FP32 on-the-fly. The embedding blob is cached; `get_embeddings()` extracts rows by byte offset.

c) Deeper details.

```
load_embeddings(path, dim):
  read file → blob; parse header; validate shape; cache metadata

get_embeddings(token_ids) → float_t[]:
  for each token_id: decode row from blob at payload + token_id *
row_bytes

decode_value dispatches:
  BF16: left-shift 16 bits → FP32 (same layout, truncated mantissa)
  FP16: re-bias exponent (15→127), shift mantissa, handle denormals
  FP32: direct memcpy
```

3. GPU Matrix Multiplication (Tiled GEMM)

a) Problem framing. Llama 3's linear projections require dense matmuls up to 4096 × 4096. A naive GPU kernel is memory-bound — each element loaded from global memory O(N) times. We need high arithmetic intensity via the shared memory hierarchy.

b) High-level solution. Thread-coarsened tiled GEMM with double-buffered shared memory. C is partitioned into 128×128 block tiles; 256 threads (16×16) iterate over K in chunks of 16. Each thread computes an 8×8 sub-tile (64 FMAs per K-step) in registers. Two shared memory buffers alternate — one feeds computation while the other loads from global memory, hiding latency.

c) Deeper details.

Data flow through the memory hierarchy:

```
Global Memory
  ↓ cooperative load: 256 threads load 128×16 (A) + 16×128 (B)
Shared Memory (double-buffered)
  smA[2][128][16+1], smB[2][16][128+1]   (+1 padding avoids bank
conflicts)
  ↓ each thread loads TM=8 from smA, TN=8 from smB
Registers: acc[8][8] += a_reg[8] × b_reg[8]
```

Double buffering loop:

```
prefetch tile 0 → buffer[0]; sync
for tile = 0 .. num_tiles-1:
    load tile+1 → buffer[nxt]          (overlapped with compute)
    for k = 0..15: acc += smA * smB    (64 FMAs, fully unrolled)
    sync; swap buffers
```

Key optimizations: B-matrix uses `float4` vectorized loads (4× fewer transactions, scalar fallback at boundaries); +1 shared memory padding breaks bank conflict stride; host wrapper manages cudaMalloc/memcpy/free with RAII cleanup.

4. CPU Matrix Multiplication Fallback

a) Problem framing. Without `nvcc`, the project must still build and run for local development.

b) High-level solution. Same `gpu_matmul` signature, triple-nested loop. Makefile auto-detects `nvcc` and links either version transparently.

c) Deeper details. Uses (i,k,j) loop order instead of textbook (i,j,k) — the innermost loop sweeps stride-1 across both B and C, yielding ~16× fewer cache misses for large matrices.

5. Weight Conversion Pipeline (Python Tooling)

a) Problem framing. Llama 3 weights ship as safetensors (JSON-indexed, variable headers). Our C++ loader needs a fixed-layout binary format.

b) High-level solution. `llama3_downloader.py` fetches shards from HuggingFace. `dumper.py` converts each tensor: one file per tensor, 280-byte header + raw BF16 payload.

c) Deeper details. Dump format:

```
Offset   Size    Field
0        256     tensor_name (null-padded ASCII)
256      4       dtype_code (uint32 LE): 0=FP32, 1=FP16, 2=BF16
260      4       ndims (uint32 LE): 1 or 2
264      8       shape[0] (uint64 LE)
272      8       shape[1] (uint64 LE, 0 for 1D)
280      var     raw payload
```

Fixed header enables single-read parsing with pointer arithmetic. One file per tensor enables OS page cache for free.

Challenges

- **BPE tokenizer correctness** — No single spec documents Llama 3's BPE. Merge order, base64 vocab, special token IDs, and byte-level fallback all had to be reverse-engineered to match HuggingFace's output.
- **BF16/FP16 numerical fidelity** — FP16-to-FP32 conversion must handle normals, denormals, and special values (inf/NaN). Small exponent re-biasing errors produce values that look close but fall

outside test tolerance.

- **CUDA tiled GEMM correctness** — Double buffering, cooperative loads, and thread-to-tile index math create partial-correctness failure modes (off-by-one tiles, uninitialized shared memory, misplaced syncs) that are hard to isolate.
- **Shared memory bank conflicts** — Without +1 padding, warp-level bank conflicts silently degrade throughput by up to 32x. Invisible in output; only detectable via `ncu` profiling.
- **float4 alignment** — Vectorized 128-bit B-matrix loads require 16-byte alignment. Bad boundary checks cause hard crashes or silent wrong reads depending on matrix dimensions.
- **No framework for intermediate validation** — With no PyTorch/TensorFlow reference, the only validation is 7 binary-fixture tests. Debugging required custom comparison tools and intermediate array dumps.
- **GCP spot VM iteration cycle** — Each change requires scp + rebuild + test on a spot T4 (~2-3 min round-trip, risk of preemption). The CPU fallback cannot catch CUDA-specific bugs.

Architecture Layers

The codebase is organized into 7 layers, ordered from lowest-level to highest:

1. Foundation

Project-wide constants and common type aliases that all other layers depend on.

File	Purpose
`config.h`	Tokenizer path, embedding dim (4096), RMS norm epsilon
`include/prelude.h`	Type aliases (`float_t`, `size_t`) and STL using-declarations

2. Interfaces & Headers

Header files defining abstract interfaces and utility types consumed by implementations.

File	Purpose
`include/tokenizer.h`	Abstract `LLMTokenizer` + concrete `BPETokenizer` class declarations
`include/loader.h`	`LlamaDumpLoader` class — mmap-based weight loading with BF16/FP16/FP32
`include/milifloat.h`	`bf16_to_float()` and `half_to_float()` bit-manipulation converters
`include/operator.cuh`	`AbstractOperator` base class for GPU operators

3. GPU Kernel Layer

CUDA kernels and CPU fallback providing the compute primitives for inference.

File	Purpose
`kernel/kernels.cuh`	Forward-declares `matmul_kernel` and `gpu_matmul` signatures

File	Purpose
`kernel/matmul.cu`	Double-buffered tiled GEMM: 128x128 blocks, float4 loads, shared memory
`kernel/matmul_cpu.cpp`	CPU fallback with cache-friendly (i,k,j) loop ordering

4. Core Implementations

C++ source implementing the tokenizer and weight loader.

File	Purpose
`src/tokenizer_bpe.cpp`	BPE encode/decode: JSON model loading, greedy merge loop, special tokens
`src/loader.cpp`	Binary weight parser: 256-byte headers, BF16 conversion, embedding extraction

5. Application Entry Point

File	Purpose
`main.cpp`	Wires together tokenizer for encode/decode demo

6. Testing

Test harness validating tokenization, embeddings, and matmul against binary fixtures.

File	Purpose	Editable?
`tests/test.cpp`	7 graded tests	**NO** (read-only)
`tests/test_api.h`	`TestAPI` interface	**NO** (read-only)
`tests/test_api.cpp`	Student bridge wiring implementations to tests	**YES**

7. Python Tooling

Offline data pipeline — no C++ dependencies at runtime.

File	Purpose
`tools/llama3_downloader.py`	Downloads Llama 3 8B from Hugging Face
`tools/dumper.py`	Converts safetensors to custom binary dump format
`tools/token_show.py`	Reference HF tokenizer for validating C++ BPE output

Key Concepts

- **BPE (Byte Pair Encoding)**: Text is tokenized by iteratively merging the highest-priority adjacent byte pairs until no merges remain. The tokenizer uses a strategy pattern — the abstract interface allows

swapping implementations.

- **Custom Binary Format**: Weights are stored as 256-byte header + BF16 payload. Faster than safetensors/GGUF because it skips JSON parsing and can be memory-mapped.
- **BFloat16**: Same exponent range as FP32 (8 bits) but only 8 mantissa bits — halves storage while maintaining dynamic range for neural network weights.
- **Double-Buffered Tiled GEMM**: The CUDA kernel overlaps loading tile N+1 from global memory while computing on tile N from shared memory, hiding latency.
- **Compile-Time GPU/CPU Selection**: The Makefile auto-detects `nvcc`. Kernel signatures in `kernels.cuh` let the linker choose either CUDA or CPU implementation — a compile-time strategy pattern.
- **Bridge Pattern (Testing)**: `test_api.cpp` is the only student-modifiable file. It connects the implementation to the professor's read-only test harness without either side knowing the other's internals.

Guided Tour

Follow these steps to understand the codebase from entry point to internals:

Step	Title	Start Here
1	**Entry Point**	`main.cpp` — see how the inference pipeline is wired
2	**Configuration**	`config.h` — model constants (embedding dim, epsilon)
3	**Common Prelude**	`include/prelude.h` — type aliases used everywhere
4	**Tokenizer Interface**	`include/tokenizer.h` — abstract + concrete class declarations
5	**Tokenizer Implementation**	`src/tokenizer_bpe.cpp` — the BPE merge loop
6	**Weight Loader Interface**	`include/loader.h` — binary weight loading API
7	**Float Conversion**	`include/milifloat.h` — BF16/FP16 to FP32 converters
8	**Weight Loader Implementation**	`src/loader.cpp` — binary parser and embedding extraction
9	**GPU Operator Base**	`include/operator.cuh` — abstract operator interface
10	**Kernel Signatures**	`kernel/kernels.cuh` — GPU/CPU matmul contract
11	**CUDA GEMM Kernel**	`kernel/matmul.cu` — the performance-critical kernel
12	**CPU Fallback**	`kernel/matmul_cpu.cpp` — builds without GPU
13	**Test Harness**	`tests/test.cpp` + `test_api.h` + `test_api.cpp` — grading bridge
14	**Python Tooling**	`tools/` — offline weight download and conversion pipeline

Complexity Hotspots

These files require the most careful attention:

File	Complexity	Why
`kernel/matmul.cu`	**Complex**	Double-buffered tiled GEMM with shared memory, float4 vectorization, bank-conflict padding
`src/loader.cpp`	**Complex**	Binary format parsing, BF16/FP16 type conversion, mmap-based embedding loading
`src/tokenizer_bpe.cpp`	**Complex**	Full BPE pipeline — JSON model loading, greedy merge loop, special token handling, base64 decoding
`tests/test.cpp`	**Complex**	7-test harness with binary fixture loading and comparison logic (read-only, but important to understand)
`tools/dumper.py`	**Complex**	Safetensors parsing, dtype normalization, per-tensor binary file output

Quick Start

```
# Build and run the main binary
make run

# Build and run all 7 tests (requires model weights in ./assets/llama3/)
make tests
for i in 1 2 3 4 5 6 7; do ./bin/tests $i; done

# Debug build
make BUILD=debug
```